Quanto sappiamo (davvero) della Dependency Injection?

Nicola Paro

Sponsor

CONSORTIVM SERVICE S.r.l.

UNIPORDENONE
moltiplica il valore della formazione

A real life story…

```
using System.Configuration;
using System.Data.SqlClient;


public class OrderService
{
    public void PlaceOrder()
    {
        var connectionString = ConfigurationManager.ConnectionStrings["OrdersDb"].ConnectionString;

        // lots of lines of logic here

        using var connection = new SqlConnection(connectionString);
        connection.Open();


        var command = connection.CreateCommand();
        command.CommandText = "INSERT INTO Orders (...) VALUES (...)";
        command.ExecuteNonQuery();


        var smtpClient = new SmtpClient("smtp.company.local");
        smtpClient.Send("noreply@company.com", "admin@company.com", "Order placed", "");
    }
}
```

A real life story…

```
using System.Configuration;
using System.Data.Sql

public class OrderSer
{
    public void Place
    {
        var connectic                                   onne                ionString;

        // lots of li

        using var connection = new SqlConnection(connectionString);
        connection.Open();

        var command =
        command.Comma                                   .)
        command.Execu

        var smtpClier                                   .loc
        smtpClient.Se                                   @cor
    }
}
```

Come lo testo?

Come lo cambio?

Come lo mantengo?

Come lo estendo?

Il problema di fondo…

Una classe che crea direttamente le proprie dipendenze gestisce **quello che usa** invece che **quello di cui ha bisogno**

Non puoi cambiare il codice implementativo in maniera semplice

Lo unit testing diventa difficile o impossibile

Una singola modifica si propaga poi in tutto il codice

Che cos’è quindi la Dependency Injection?

E’ una tecnica e design pattern in cui gli oggetti non creano le proprie dipendenze, ma le dipendenze sono fornite dall’esterno.

Depend on abstractions, not implementations

Che cos’è quindi la Direct Injection?

Che cos’è quindi la Dependency Injection?

```
public interface IEngine { }
public class VolksWagenEngine : IEngine { }
public class BMWEngine : IEngine { }
public class FerrariEngine : IEngine { }


public interface IFarmingTractor { }
public class FarmingTractor(IEngine engine) : IFarmingTractor { }


public class Farmer(IFarmingTractor tractor) { }
```

Che cos’è un DI Container?

Un DI Container è una libreria che gestisce automaticamente le dipendenze tra oggetti in un’applicazione.

- ✓ Risolve le dipendenze
- ✓ Gestisce il ciclo di vita delle dipendenze
- ✓ Supporta costruttori complessi
- ✓ Facilita test e manutenzione
- ✓ Riduce l’accoppiamento

- ✕ Non è un Service Locator
- ✕ Non decide la logica dell’app
- ✕ Non sostituisce una buona progettazione

DI Containers

.NET

Autofac

SIMPLE INJECTOR™

Castle Windsor

DI Containers

.NET

Microsoft.Extensions.DependencyInjection

- Container DI integrato in .NET
- Approccio consistente tra framework diversi
- Leggero e veloce
- Funziona con ASP.NET Core, WindowsServices, Console apps, Windows Forms ecc...
- **netstandard 2.0**
  - Funziona sia con .NET 8, 9, 10 che con .NET Framework

IServiceCollection ed IServiceProvider

- **IServiceCollection** ha il compito di definire come costruire l'albero delle dipendenze
- **IServiceProvider** ha il compito di
  - Creare le istanze degli oggetti
  - Iniettare le dipendenze
  - Gestire il ciclo di vita degli oggetti secondo il lifetime che specifichiamo in fase di registrazione in IServiceCollection

Service Lifetime

Singleton	Scoped	Transient
• Singola istanza per l'intera applicazione • Deve essere thread-safe	• Istanza per ogni scope*	• Nuova istanza ogni volta che è richiesto • Pensato per servizi leggeri e senza stato

* In AspNetCore:

- Per ogni richiesta HTTP, viene creato uno scope in automatico dal framework.
- Non è vero che ad uno scope corrisponde sempre una richiesta HTTP.

Come avviene l’iniezione

L’iniezione dei servizi avviene principalmente tramite Constructor Injection, in cui le dipendenze sono iniettate attraverso i parametri del costruttore:

- Dipendenze esplicite
- Previene situazioni in cui esistono istanze parzialmente inizializzate

Come registro una dipendenza?

Astrazione + Implementazione

```
var services = new ServiceCollection();
```

Lifetime	Abstraction	Implementation

```
services.AddSingleton<IMyService1, MyService1Implementation>();
services.AddScoped<IMyService2, MyService2Implementation>();
services.AddTransient<IMyService3, MyService3Implementation>();
```

Come registro una dipendenza?

Astrazione + Implementazione

```
var services = new ServiceCollection();



services.AddSingleton<IMyService1, MyService1Mock>();

services.AddScoped<IMyService2, MyService2Mock>();

services.AddTransient<IMyService3, MyService3Implementation>();
```

Come registro una dipendenza?

Solo implementazione

```
var services = new ServiceCollection();



services.AddSingleton<MyService1Implementation>();

services.AddScoped<MyService2Implementation>();

services.AddTransient<MyService3Implementation>();
```

Come registro una dipendenza?

Astrazione + implementationFactory per l'implementazione

```
var services = new ServiceCollection();


services.AddSingleton<IMyService1>(sp => new MyService1Implementation());

services.AddScoped<IMyService2>(sp => new MyService2Implementation());

services.AddTransient<IMyService3>(sp => new MyService3Implementation());
```

sp è un IServiceProvider

Come registro una dipendenza?

Solo implementationFactory

```
var services = new ServiceCollection();


services.AddSingleton(sp => new MyService1Implementation());

services.AddScoped(sp => new MyService2Implementation());

services.AddTransient(sp => new MyService3Implementation());
```

sp è un IServiceProvider

Come registro una dipendenza?

Quando ha senso usare la factory?

```
public interface IMyServiceA { }
public interface IMyServiceB { }

public class MyServiceAImplementation(string name) : IMyServiceA { }
public class MyServiceBImplementation(IMyServiceA serviceA, string name) : IMyServiceB { }


services.AddSingleton<IMyServiceA>(
    sp => new MyServiceAImplementation("ServiceAName"));

services.AddSingleton<IMyServiceB> (
    sp => new MyServiceBImplementation(sp.GetRequiredService<IMyServiceA>(), "ServiceBName"));
```

Come registro una dipendenza?

Quando un servizio ha tante dipendenze…

```
public class MyService(IMyServiceA serviceA, IMyServiceB serviceB
    , IMyServiceC serviceC, IMyServiceD serviceD
    , IMyServiceE serviceE) : IMyService { }

services.AddSingleton<IMyServiceA, MyServiceAImplementation>();
services.AddSingleton<IMyServiceB, MyServiceBImplementation>();
services.AddSingleton<IMyServiceC, MyServiceCImplementation>();
services.AddSingleton<IMyServiceD, MyServiceDImplementation>();
services.AddSingleton<IMyServiceE, MyServiceEImplementation>();
services.AddSingleton<IMyService, MyService>();
```

Come registro una dipendenza?

Quando un servizio ha tante dipendenze, ma mi serve la factory…

```
public class MyService(IMyServiceA serviceA, IMyServiceB serviceB
    , IMyServiceC serviceC, IMyServiceD serviceD
    , IMyServiceE serviceE, string someString) : IMyService { }

services.AddSingleton<IMyServiceA, MyServiceAImplementation>();
services.AddSingleton<IMyServiceB, MyServiceBImplementation>();
services.AddSingleton<IMyServiceC, MyServiceCImplementation>();
services.AddSingleton<IMyServiceD, MyServiceDImplementation>();
services.AddSingleton<IMyServiceE, MyServiceEImplementation>();
services.AddSingleton<IMyService>(sp => new MyService(
    sp.GetRequiredService<IMyServiceA>(), sp.GetRequiredService<IMyServiceB>(),
    sp.GetRequiredService<IMyServiceC>(), sp.GetRequiredService<IMyServiceD>(),
    sp.GetRequiredService<IMyServiceE>(),"SomeStringValue"
));
```

Come registro una dipendenza?

Quando un servizio ha tante dipendenze, ma mi serve la factory…

```
public class MyService(IMyServiceA serviceA, IMyServiceB serviceB
    , IMyServiceC serviceC, IMyServiceD serviceD
    , IMyServiceE serviceE, string someString) : IMyService { }

services.AddSingleton<IMyServiceA, MyServiceAImplementation>();
services.AddSingleton<IMyServiceB, MyServiceBImplementation>();
services.AddSingleton<IMyServiceC, MyServiceCImplementation>();
services.AddSingleton<IMyServiceD, MyServiceDImplementation>();
services.AddSingleton<IMyServiceE, MyServiceEImplementation>();
services.AddSingleton<IMyService>(sp =>
    ActivatorUtilities.CreateInstance<MyService>(sp, "SomeStringValue");
```

ActivatorUtilities.CreateInstance

ActivatorUtilities.CreateInstance

Normalmente, il container DI crea automaticamente le istanze risolvendo tutte le dipendenze dal servizio registrato.

Tuttavia, a volte potremmo voler creare un'istanza di una classe dove solo alcuni parametri del costruttore sono gestiti dal container, mentre altri li passiamo esplicitamente.

- i valori in parameters vengono abbinati in ordine ai parametri del costruttore che non possono essere risolti dal container.
- viene scelto il costruttore pubblico con il maggior numero di parametri che può essere soddisfatto (combinando servizi risolti + parametri forniti).
- Se nessun costruttore è compatibile → InvalidOperationException.
- Utile quando non vuoi registrare un tipo nel container, ma comunque sfruttare il DI per alcune sue dipendenze.

Ma… Ora ho alcune domande:

Come uso questa cosa nel mio applicativo legacy?

Demo

Principali errori

- Service Locator
  - Iniettare IServiceProvider ovunque
  - Nasconde le reali dipendenze e rende meno chiaro il codice.
- Captive Dependency
  - Singleton che dipende da un servizio Scoped o Transient
- Usare troppi Singleton
  - Possono causare memory leak
  - Stato condiviso tra più servizi

Best Practices

- Preferire le constructor injection
- Registra i servizi vicino alla composition root
- Servizi piccoli e focalizzati sul loro obiettivo
  - OrderRepository gestisce gli ordini, non crea un SmtpClient per mandare una mail
- Astrazioni chiare
  - IOrderRepository non definisce un metodo SendOrderConfirmationEmail
- Evita le dipendenze statiche
  - JsonSerializer
  - Console

Ricapitolando:

- La Dependency Injection resolve il problema delle dipendenze dirette
- E’ focalizzata sulle astrazioni
- Microsoft.Extensions.DependencyInjection è semplice e potente

Grazie!

Domande?

👇 Contatti 👇

Nicola Paro

Senior Software Engineer | Solution Architect

Microsoft® MVP Most Valuable Professional

linkedin.com/in/nicolaparo